AVIZ ACADEMY

AWS & DevOps Training

Jenkins Training Series

Jenkins Pipelines

Declarative | Scripted | Post Actions | Approval Gates

💡	*Real-Life Analogy: A Jenkins Pipeline is like a car assembly line in a factory. Raw parts (source code) come in one end, pass through welding, painting, testing, and quality-check stations (stages), and a finished car (deployable artifact) rolls out the other end — automatically, every single time.*

avizacademy.com

01 What is a Jenkins Pipeline?

A Jenkins Pipeline is a suite of plugins that supports implementing and integrating Continuous Delivery (CD) pipelines into Jenkins. It defines your entire build-test-deploy process as code, stored alongside your application source code in a file called Jenkinsfile.

💡	*Think of it like a recipe card. Without a recipe, every cook makes the dish differently. With a Jenkins Pipeline (the recipe), every build follows the exact same steps in the exact same order — no surprises, no human errors.*

Why Pipeline as Code?

- Version controlled — stored in Git alongside your source code
- Reproducible — same steps, every run, every environment
- Reviewable — team members can review pipeline changes via pull requests
- Shareable — one pipeline definition can be reused across projects

02 Declarative vs Scripted Pipelines

Jenkins supports two syntaxes for writing pipelines. Declarative is modern, structured, and recommended for most use cases. Scripted is older, Groovy-based, and offers more flexibility for complex logic.

Declarative Pipeline	Scripted Pipeline
Introduced in Jenkins 2.x (recommended)	Original pipeline syntax
Structured, opinionated syntax	Full Groovy scripting freedom
Starts with pipeline { }	Starts with node { }
Easier to read and write	More flexible but complex
Built-in error handling with post { }	Manual error handling required
Best for standard CI/CD workflows	Best for complex conditional logic

Declarative Pipeline — Basic Structure

💡	*Think of Declarative Pipeline like filling out an official government form. There are fixed sections you must fill — agent, stages, steps — and you fill in your details. Clean, structured, predictable.*

```
pipeline {
    agent any           // Run on any available Jenkins agent
    stages {            // The stages container
        stage('Build') {
            steps {
                echo 'Compiling code...'
            }
        }
        stage('Test') {
            steps {
                echo 'Running tests...'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying application...'
            }
        }
    }
}
```

Scripted Pipeline — Basic Structure

💡	*Scripted Pipeline is like a blank notebook. You write whatever you want in Groovy. Maximum flexibility, but you need to know what you're doing.*

```
node {                      // Run on any available node
    stage('Build') {
        echo 'Building...'
    }
    stage('Test') {
        echo 'Testing...'
    }
    stage('Deploy') {
        echo 'Deploying...'
    }
}
```

03 Key Syntax Elements — Quick Reference

Understanding the building blocks of a Jenkins Pipeline is essential before writing one. Here is your quick-reference card for the most important keywords.

Keyword	Purpose	Real-Life Analogy
`pipeline { }`	Defines the entire pipeline	*The factory building itself*
`agent any`	Run on any available agent/node	*Assign task to any available worker*
`stages { }`	Container for all stages	*The list of departments in the factory*
`stage('Name') { }`	One named phase of the pipeline	*One department: Welding, Painting, QA*
`steps { }`	The actual commands to run	*The specific tasks a worker performs*
`echo`	Print message to console	*Worker announcing progress over intercom*
`sh 'command'`	Run a shell command	*Worker executing a physical task*
`git url: '...'`	Checkout code from Git	*Picking up raw materials from warehouse*
`post { }`	Actions after pipeline finishes	*End-of-day cleanup and reporting*
`input`	Pause for human approval	*Manager signing off before shipping*

04 Pipeline Examples — From Simple to Real World

Example 1: Hello World Pipeline

The simplest possible pipeline — perfect for verifying your Jenkins setup works. Think of this as turning the factory lights on for the first time.

```
pipeline {
    agent any
    stages {
        stage('Hello') {
            steps {
                echo 'Hello, Jenkins Pipeline!'
            }
        }
    }
}
```

- One stage, one step — bare minimum for a valid pipeline
- Use this to confirm Jenkins is configured correctly before adding complexity

Example 2: Multiple Stages — The CI/CD Assembly Line

This is the classic pattern you'll use in real projects. Three stages: Build, Test, Deploy — each with its own responsibility, running sequentially.

> 💡 *This is your factory assembly line: Raw materials enter (code checked out), get assembled (Build), inspected by QA (Test), and shipped to the customer (Deploy). Each station only starts after the previous one passes.*

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building the application...'
            }
        }
        stage('Test') {
            steps {
                echo 'Running tests...'
            }
        }
        stage('Deploy') {
            steps {
```

```
                echo 'Deploying application...'
            }
        }
    }
}
```

Example 3: Running Shell Commands

Real pipelines run actual system commands using the sh step. You can run any Linux command that you would normally type in a terminal.

```
pipeline {
    agent any
    stages {
        stage('Hello') {
            steps {
                sh 'uname'    // Prints OS name
            }
        }
    }
}
```

Run multiple shell commands with triple-quote blocks:

```
sh '''
    ls
    lsblk
    df -Th
    hostname
'''
```

Example 4: Code Checkout from GitHub (Syntax Generator)

Stage 1 runs hostname to show which agent is running. Stage 2 uses the Jenkins Git plugin to pull code from GitHub.

```
pipeline {
    agent any
    stages {
        stage('stage-1') {
            steps {
                sh 'hostname'   // Confirm which agent is running
            }
        }
        stage('stage-2') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/avizway1/awar04-jenkins.git'
            }
        }
    }
}
```

Example 5: Multiple Steps in One Stage (Pipeline as Code)

A single stage can contain multiple steps — they run sequentially. This is useful when related tasks belong together logically.

```
pipeline {
    agent any
    stages {
        stage('Hello') {
            steps {
                sh 'uname'                  // Step 1: OS info
                echo 'second step'          // Step 2: Log message
                git branch: 'main', url:
'https://github.com/avizway1/awar04-jenkins.git'
            }
        }
    }
}
```

Example 6: Real-World CI/CD — Git + Maven Build + Tests

💡	*Complete factory workflow: Truck delivers raw steel (git clone) → Workers build the car body (mvn clean package) → QA team inspects every weld (mvn test) → Only a car that passes inspection ships to the next stage.*

```
pipeline {
    agent any
    stages {
        stage('Checkout') {
            steps {
                sh 'git clone https://github.com/avizway1/awar04-jenkins.git'
            }
        }
        stage('Build') {
            steps {
                sh 'echo "Compiling code..."'
                sh 'mvn clean package'   // Compile + test + package into WAR/JAR
            }
        }
        stage('Test') {
            steps {
                sh 'mvn test'   // Run full test suite
            }
        }
    }
}
```

05 Post Actions — What Happens After the Pipeline?

The post section defines actions that run after all stages complete. It is your pipeline's cleanup crew and notification system — always runs regardless of whether the build passed or failed.

💡	*Think of post as the factory shift-end protocol. Whether it was a successful production day (success) or a machine broke down (failure), the team still fills out the daily report, turns off the lights, and locks the doors (always).*

Condition	When It Runs
success	Only when ALL stages passed
failure	Only when a stage failed
always	Always — success or failure
unstable	Build completed but tests failed (yellow)
aborted	Build was manually cancelled
changed	Result differs from the previous build

Example 7: Pipeline with Post Actions

```
pipeline {
    agent any
    stages {
        stage('Build') {
            steps {
                sh 'echo "Building project..."'
                sh 'sleep 5'   // Simulate build time
            }
        }
        stage('Test') {
            steps {
                sh 'echo "Running tests..."'
                sh 'exit 1'   // Force failure to demo post behavior
            }
        }
    }
    post {
        success {
            echo 'Build SUCCEEDED! Ready to deploy.'
        }
        failure {
            echo 'Build FAILED! Check the logs above.'
        }
        always {
```

```
            echo 'Pipeline finished. Performing cleanup...'
        }
    }
}
```

Note: sh 'exit 1' intentionally forces a failure to demonstrate the failure and always blocks. Remove it in a real pipeline.

06 Manual Approval — Human in the Loop

The input step pauses the pipeline and waits for a human to click a button before continuing. This is critical for production deployments — you want a human to review and approve before code goes live.

> 💡 *Real life: Before a flight takes off, the captain, first officer, and air traffic controller all give clearance. The plane (pipeline) only moves forward once all humans say YES. This is exactly what the Jenkins input step does for your production deployments.*

Example 8: Approval Gate — Adds Approval Process

```
pipeline {
    agent any
    stages {
        stage('Confirmation') {
            steps {
                script {
                    input message: 'Do you want to continue?', ok: 'Yes'
                }
            }
        }
        stage('Next Step') {
            steps {
                echo 'Pipeline continued after approval'
            }
        }
    }
}
```

Example 9: Full Approval Gate with Timeout

In production you don't want a pipeline paused forever. Wrap input with a timeout so the build auto-aborts if no one approves within the allotted time:

```
pipeline {
    agent any
    stages {
        stage('Build') { steps { echo 'Build complete.' } }
        stage('Test')  { steps { echo 'All tests passed.' } }
        stage('Approval Gate') {
            steps {
                timeout(time: 30, unit: 'MINUTES') {  // Auto-abort after 30 min
                    script {
                        input message: 'Deploy to PRODUCTION?',
```

```
                                ok: 'Approve Deployment',
                                submitter: 'admin'   // Restrict who can approve
                            }
                        }
                    }
                }
                stage('Deploy to Production') {
                    steps {
                        echo 'Deploying to Production environment...'
                    }
                }
            }
        }
```

Key Points about input:

- Pipeline shows as 'Paused' in Jenkins — not failed, not passed
- The submitter parameter restricts which users/groups can approve
- Always wrap with timeout() in production to prevent infinite pausing
- Great for dev→staging→production promotion workflows

07 Scripted Pipelines — When You Need More Power

While Declarative is recommended for most cases, Scripted pipelines give you the full power of Groovy — conditions, loops, exception handling, and custom functions.

Example 10: Scripted Pipeline — Single Stage

```
node {
    stage('Build') {
        sh 'echo "hello from avinash"'
    }
}
```

Example 11: Scripted Pipeline — Multiple Stages

```
node {
    stage('Build') {
        echo 'Building...'
    }
    stage('Test') {
        echo 'Testing...'
    }
    stage('Deploy') {
        echo 'Deploying...'
    }
}
```

Bonus: Scripted Pipeline with Groovy Conditions

The real power of Scripted pipelines is using full Groovy — if/else conditions, loops, try/catch:

```
node {
    def BUILD_ENV = 'staging'   // Define a variable

    stage('Build') {
        sh 'mvn clean package'
    }

    stage('Deploy') {
        if (BUILD_ENV == 'production') {
            echo 'Deploying to PRODUCTION...'
        } else {
            echo "Deploying to ${BUILD_ENV}..."
```

```
            }
        }
    }
```

08 Day 4 Summary — What You Learned

Congratulations on completing Day 4! Here is a full recap of everything covered today:

Topic	Key Takeaway
`What is a Pipeline`	Automated Build-Test-Deploy process defined as code in a Jenkinsfile
`Declarative vs Scripted`	Declarative = structured & recommended; Scripted = full Groovy power
`pipeline { }`	The outer container for all declarative pipeline code
`agent any`	Run the pipeline on any available Jenkins node/agent
`stages { } + stage( )`	Logical phases of your pipeline (Build, Test, Deploy)
`steps { }`	The actual commands to execute within each stage
`sh '...' and echo`	Run shell commands or print messages in pipeline steps
`git url:`	Checkout source code from a Git repository
`Multiple steps per stage`	A single stage can contain many sequential step commands
`post { }`	Conditional actions after all stages: success, failure, always
`input step`	Pause pipeline and wait for human approval before continuing
`timeout wrapper`	Auto-abort if no approval received within a time limit

Quick Maven Commands Reference

```
mvn validate          # Check project structure is correct
mvn compile           # Compile source code only
mvn test              # Compile + run unit tests
mvn package           # Compile + test + create WAR/JAR
mvn clean install     # Full clean build + install to local repo
mvn clean package -DskipTests   # Skip tests (faster, use cautiously)
```

LAB Hands-On Lab Exercises

Complete these exercises in order. Each lab builds on the previous one.

Lab 1: Your First Pipeline

1. Go to Jenkins Dashboard → New Item → Pipeline
2. Name it: my-first-pipeline
3. In the Pipeline section, paste the Hello World example
4. Click Build Now and verify the console shows 'Hello, Jenkins Pipeline!'
5. Add a second stage called 'Goodbye' that echoes 'Session complete!'

Lab 2: Multi-Stage with Shell Commands

6. Create a new Pipeline job called: shell-pipeline
7. Write a 3-stage pipeline: System-Info, Disk-Check, Network-Check
8. System-Info: run uname, hostname, whoami
9. Disk-Check: run df -Th and lsblk
10. Network-Check: run hostname -I

Lab 3: Git Checkout + Maven Build

11. Create a pipeline that checks out code from your GitHub repo
12. Add a Build stage that runs: mvn clean package
13. Add a Test stage that runs: mvn test
14. Run the pipeline and verify the .war file appears in the target/ directory

Lab 4: Post Actions

15. Take your Lab 3 pipeline and add a post section
16. On success: echo a success message
17. On failure: echo a failure message
18. Always: echo 'Pipeline execution complete'
19. Test both paths — use sh 'exit 1' to force a failure

Lab 5: Approval Gate

20. Create a pipeline: Build → Approval → Deploy
21. Approval stage uses the input step with a custom message
22. Wrap input with a 10-minute timeout
23. Run the pipeline, observe it pause, then click the approval button
24. Verify the Deploy stage runs only after approval

AVIZ ACADEMY | AWS & DevOps Training | avizway.com